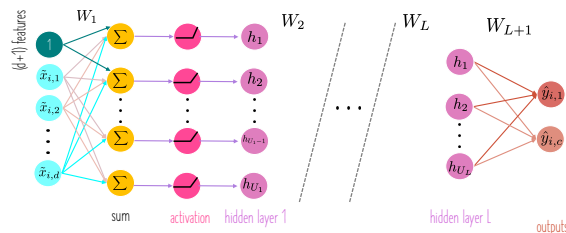
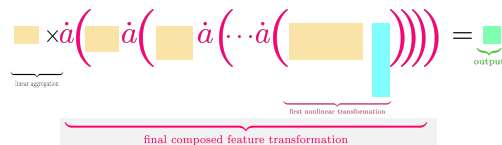


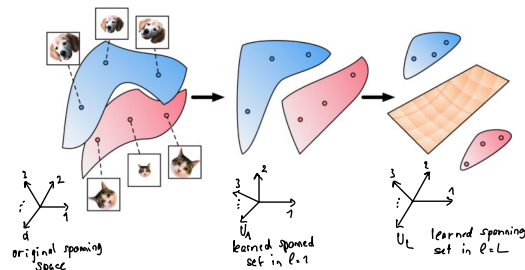
<https://github.com/basiralab>

Feature embedding

Feature embedding in deep learning and manifold learning

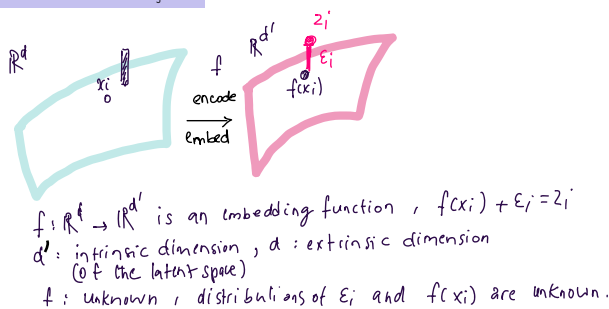


With higher computational budget we achieve the Da Vinci principle "Simplicity is the Ultimate Sophistication"



😊 The power of DL models lies in learning how to embed complex features sequentially into manifolds (subspaces to learn) that decrease in complexity until 'finding' an ultimate low-dimensional representation space (a much simpler manifold) where linear models can work well.

What is manifold learning?



- ⊗ Underlying reductionist hypothesis: neighborhood preservation across the original and embedding spaces
- ⊗ The neighborhood in the original domain may be highly noisy (outliers in the local neighborhood of a sample)

The evolving landscape of feature embedding

ML problem formalization

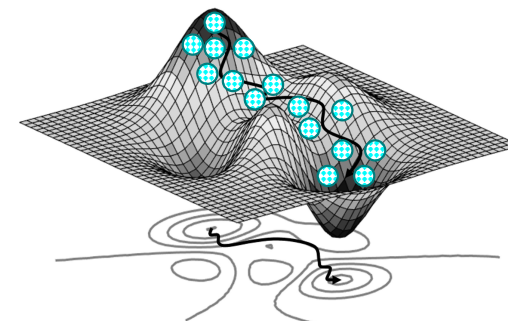
$$l(w) = \frac{1}{n} \sum_{i=1}^n (\tilde{x}_i^T w - y_i)^2$$

$\begin{matrix} (d+1) \text{ features} \\ n \text{ samples} \end{matrix} \times \begin{matrix} (d+1) \text{ features} \\ w \in \mathbb{R}^{d+1} \end{matrix} = \begin{matrix} n \text{ samples} \\ \hat{y} \in \mathbb{R}^n \end{matrix} \approx \begin{matrix} n \text{ samples} \\ y \in \mathbb{R}^n \end{matrix}$

We learn a shared mapping w across all training samples. However, our training samples are not allowed to directly interact.

Underlying reductionist assumption

Individualistic points -- data samples live "independently" on a high-dimensional manifold.



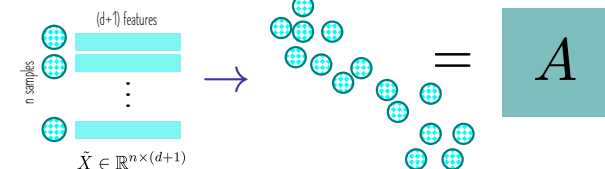
Even in manifold learning, we learn how to calculate the similarity between points but not how to connect them. We are one step away from that!

- ⊗ Cannot utilize node, edge and graph features.
- ⊗ Such approaches cannot obtain embeddings for nodes not in the training set. If we add a node, we need to recompute all node feature representations (embeddings).

1. Node-based features

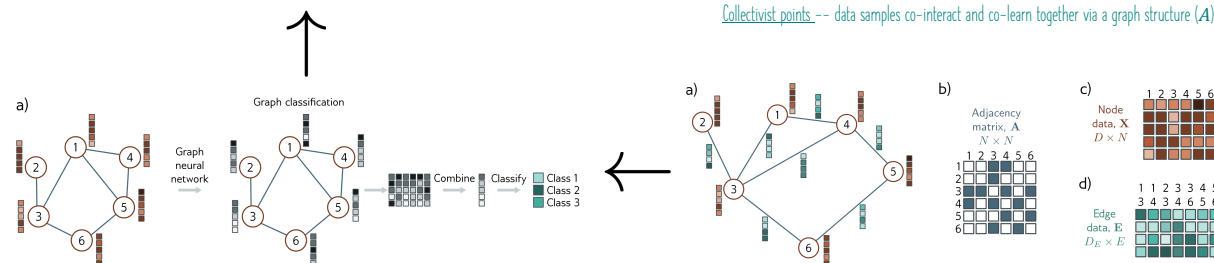
2. Edge-based features

3. Graph-based features



Use a graph to model and capture the relationships between pairs of data points. Each sample interacts with all other samples.

Classical node, edge and graph-based feature extraction methods



How to get the node features?
 How to get the edge or graph features?

We can also add edge features. Remember you've got a graph to learn on!

A single GCN layer (shallow)

Classical node encoding/embedding (with no features)

A vanilla GCN layer

Training sample (node) mapping/encoding through a deep neural network

Graph mapping/embedding through a graph neural network

$$H_{k+1} = \underbrace{f(H_k, A)}_{\text{generalized propagation rule}}$$

Our goal: Shifting from an "individualistic" to a "collectivist" learning paradigm through the graph matrix $\mathbf{A}$

A diagram illustrating a sequence of blue circles with a cross pattern, followed by an equals sign and a teal square containing the letter A .

$$l(w) = \frac{1}{n} \sum_{i=1}^n (\tilde{x}_i^T w - y_i)^2$$

Deep (sequential) node
representation learning

$$\begin{aligned} \mathbf{H}_1 &= \mathbf{F}[\mathbf{X}, \mathbf{A}, \phi_0] \\ \mathbf{H}_2 &= \mathbf{F}[\mathbf{H}_1, \mathbf{A}, \phi_1] \\ \mathbf{H}_3 &= \mathbf{F}[\mathbf{H}_2, \mathbf{A}, \phi_2] \\ &\vdots \\ \mathbf{H}_K &= \mathbf{F}[\mathbf{H}_{K-1}, \mathbf{A}, \phi_{K-1}], \end{aligned}$$

 Sample embedding = latent/hidden representation = encoding = transformation = mapping

learn a node embedding that encodes its network structure.


 Graph $G = (V, E)$ is mapped to a node representation $z_u \in \mathbb{R}^d$ via an encoding function $f(u) = z_u$. This representation is a powerful node representation that can boost downstream learning tasks (e.g., classification).

Constrained embedding

↓
preserve the local structure / neighborhood
of the graph.

$$\max f(u)^T f(v) = z_v^T z_v$$

Diagram illustrating the encoding of a graph $G = (V, E)$ into a graph $G' = (V', E')$. The graph G has nodes u and v . The graph G' has nodes u' and v' . The mapping f maps nodes of G to nodes of G' . The mapping f maps edges of G to edges of G' . The mapping f maps the graph G to the graph G' . The mapping f maps the graph G to the graph G' .

$$\text{similarity}(u, v) \approx z_u^T z_v$$

↓
objective : $\begin{cases} \text{maximize } f(u)^T f(v) = z_u^T z_v \\ \text{for pairs } (u,v) \text{ that are close / similar} \end{cases}$

Diagram illustrating the encoding process. A graph $G = (V, E)$ is shown on the left. An arrow labeled "encode" points from the graph to a pink box labeled Z , which represents the embedding matrix.

```

graph TD
    A([How to define node similarity?]) --- B[are connected]
    A --- C[share neighbors]
    A --- D[have similar to biological roles]
  
```

* TRUE OR FALSE :

- 1) Shallow node embedding is a task-dependent learning problem.
- 2) Node embedding is unsupervised or self-supervised.
- 3) We utilize node labels and features for embedding

Graph Convolutional Networks (UDL-13)

Introducing layer-wise non-linearity and biases into GCN

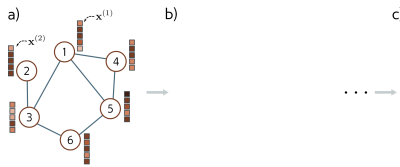


Figure Simple Graph CNN layer. a) Input graph consists of structure (embodied in graph adjacency matrix $\mathbf{A}$, not shown) and node embeddings (stored in columns of $\mathbf{X}$).

Node embedding update rule

message (embedding) aggregation across neighbors

$$\text{agg}[n, k] = \sum_{m \in \text{Ne}[n]} \mathbf{h}_k^{(m)}$$

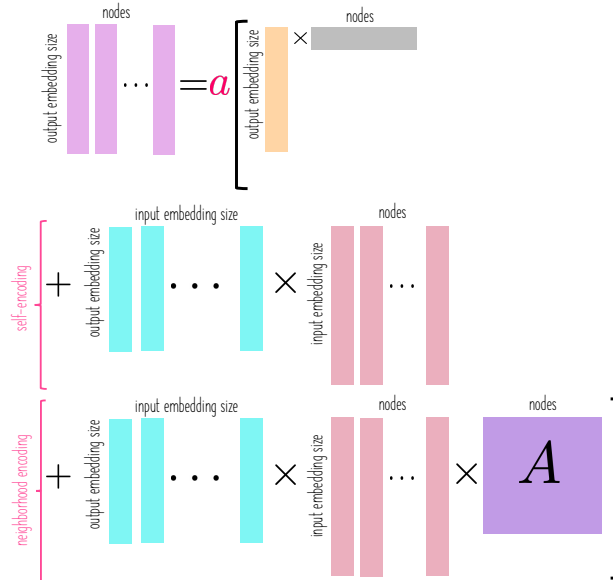
$$\mathbf{h}_{k+1}^{(n)} = \mathbf{a} \left[\beta_k + \Omega_k \cdot \mathbf{h}_k^{(n)} + \Omega_k \cdot \text{agg}[n, k] \right]$$

learned embedding of node n in layer $(k+1)$

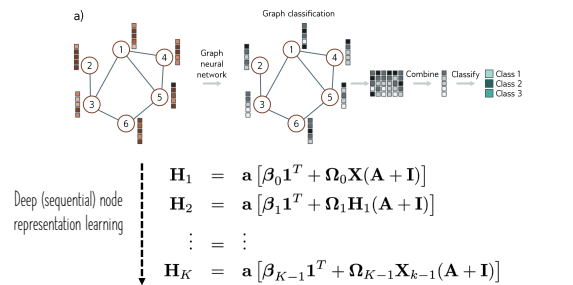
$$\text{output embedding size} = \mathbf{a} \left[\text{output embedding size} + \text{input embedding size} \times \text{input embedding size} + \text{input embedding size} \times \text{input embedding size} \right]$$

Update rule for all nodes

$$\begin{aligned} \mathbf{H}_{k+1} &= \mathbf{a} \left[\beta_k \mathbf{1}^T + \Omega_k \mathbf{H}_k + \Omega_k \mathbf{H}_k \mathbf{A} \right] \\ &= \mathbf{a} \left[\beta_k \mathbf{1}^T + \Omega_k \mathbf{H}_k (\mathbf{A} + \mathbf{I}) \right], \end{aligned}$$



Example: graph classification



$$\text{Network output function (output prediction)} \quad \mathbf{f}[\mathbf{X}, \mathbf{A}, \Phi] = \text{sig}[\beta_K + \omega_K \mathbf{H}_K \mathbf{1}/N]$$

$$y = \text{sig}[\beta_K + \omega_K \mathbf{H}_K \mathbf{1}/N]$$

$$\text{learned weights} \times \text{embedding size} \times \text{nodes} \times \text{"1" vector} \frac{1}{N} = \text{output}$$

Going deeper by adding more layers

How to combine the current node and aggregated neighbors?

Simply sum them up

$$\mathbf{H}_{k+1} = \mathbf{a} \left[\beta_k \mathbf{1}^T + \Omega_k \mathbf{H}_k (\mathbf{A} + \mathbf{I}) \right]$$

Add a diagonal enhancement

$$\mathbf{H}_{k+1} = \mathbf{a} \left[\beta_k \mathbf{1}^T + \Omega_k \mathbf{H}_k (\mathbf{A} + (1 + \epsilon_k) \mathbf{I}) \right]$$

Going deeper with more layers

Treat the current node differently from the aggregated neighbors by applying a different linear transform Ψ_k to the current node:

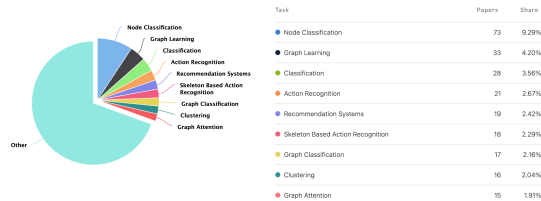
$$\begin{aligned} \mathbf{H}_{k+1} &= \mathbf{a} \left[\beta_k \mathbf{1}^T + \Psi_k \mathbf{H}_k + \Omega_k \mathbf{H}_k \mathbf{A} \right] \\ &= \mathbf{a} \left[\beta_k \mathbf{1}^T + [\Omega_k \quad \Psi_k] \begin{bmatrix} \mathbf{H}_k \mathbf{A} \\ \mathbf{H}_k \end{bmatrix} \right] \\ &= \mathbf{a} \left[\beta_k \mathbf{1}^T + \Omega'_k \begin{bmatrix} \mathbf{H}_k \mathbf{A} \\ \mathbf{H}_k \end{bmatrix} \right], \end{aligned}$$

Graph Convolutional Networks (UDL-13)

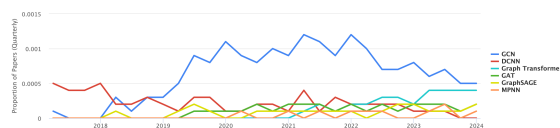
GCNs are hot topics!

Papers with code

Tasks



Usage Over Time



© This feature is experimental, we are continuously improving our matching algorithm.

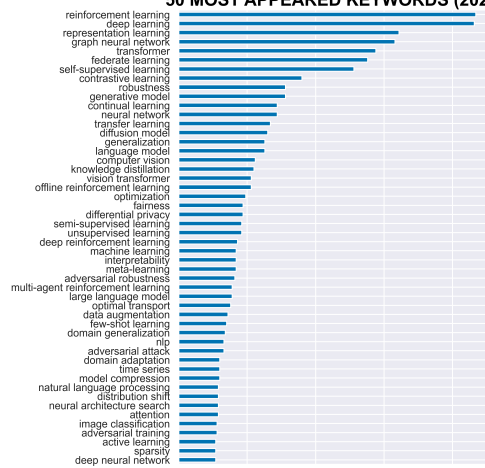
<https://paperswithcode.com/method/gcn>

ICLR 2023 open review data

• Top-10 Ranking between 2022 and 2023 (full list please refer to [keywords.md](#))

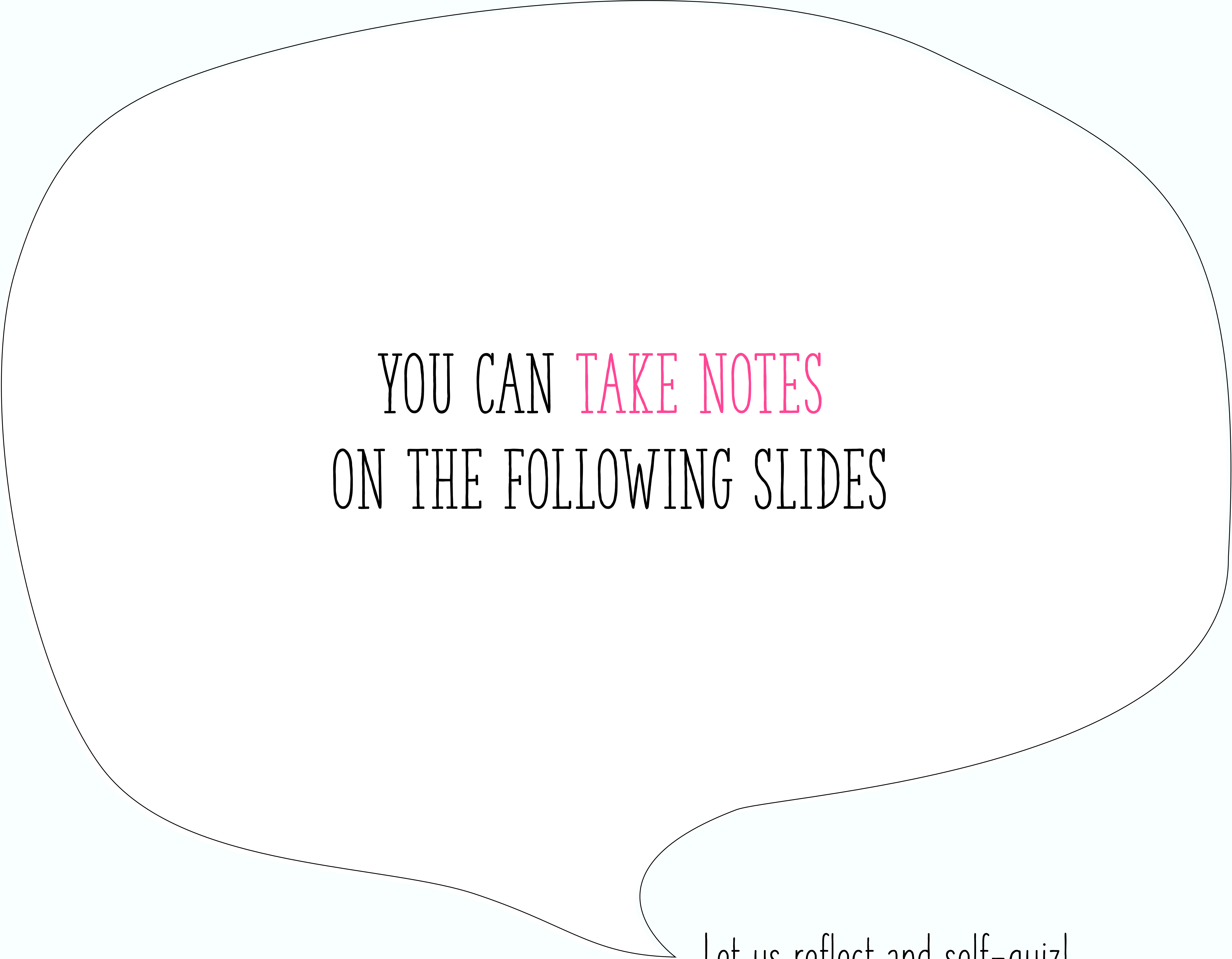
Keyword	2022	2023
reinforcement learning	1	1
deep learning	2	2
representation learning	4	3
graph neural network	3	4
transformer	5	5
federate learning	7	6
self-supervised learning	6	7
contrastive learning	10	8
robustness	9	9
generative model	8	10

50 MOST APPEARED KEYWORDS (2023)



<https://github.com/EdoardoCecchi/ICLR2023-OpenReviewDataTables-machine-ss-file>

Recall box: summarize what you remember from this lecture below



YOU CAN TAKE NOTES
ON THE FOLLOWING SLIDES

Let us reflect and self-quiz!

Preliminaries: machine & deep learning, feature extraction and graphs

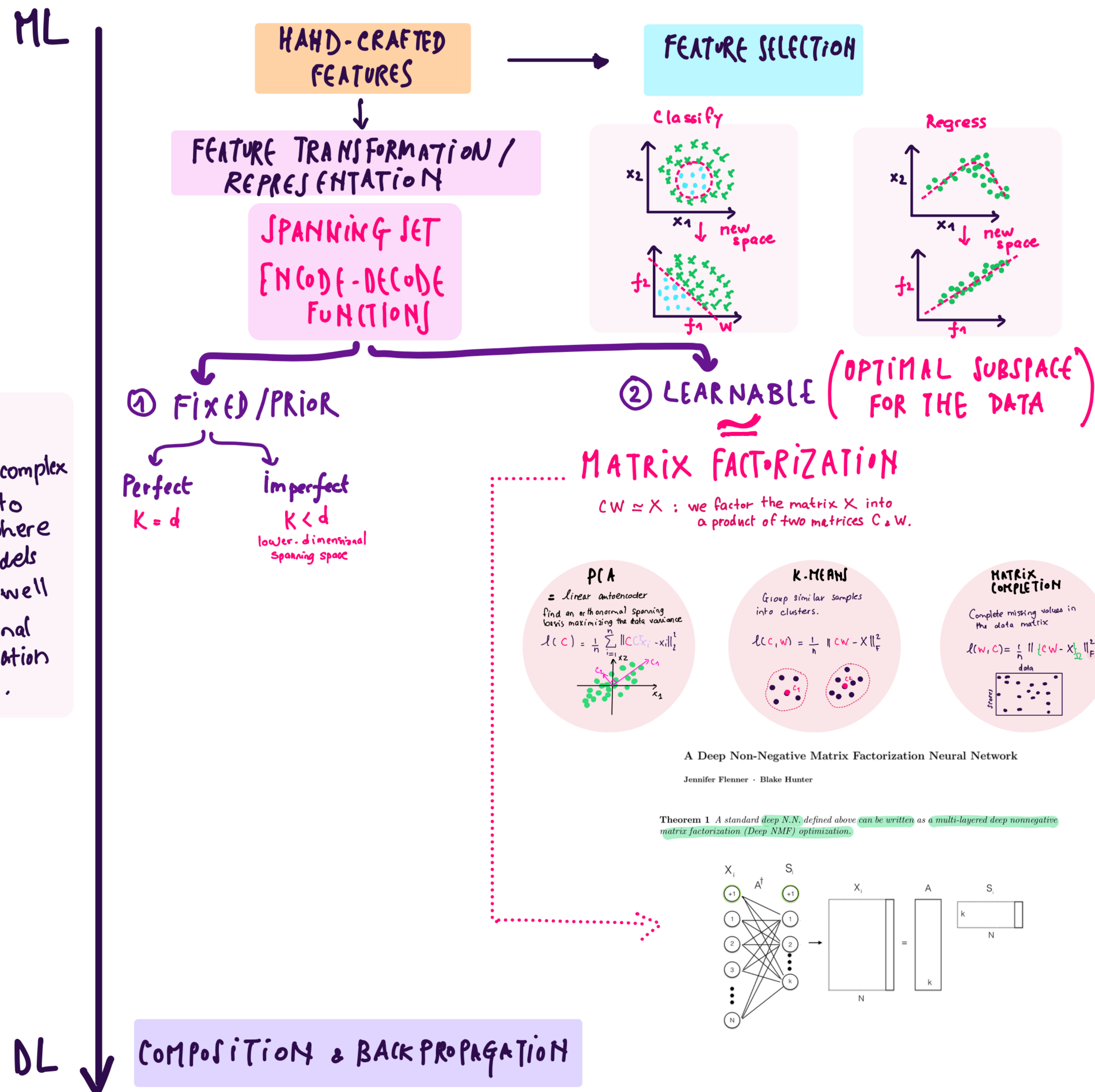
Preliminaries

Personal reflections and notes

- Intuitive Deep Learning Series
BASIRA Lab - 10 / 23
- 1 [Intuitive Deep Learning] 1.1 Supervised linear and non...
 - 2 [Intuitive Deep Learning] 1.2 Supervised linear and...
 - 3 [Intuitive Deep Learning] 1.3 Supervised linear and non...
 - 4 [Intuitive Deep Learning] 1.4 Spanning sets & matrix...
 - 5 [Intuitive Deep Learning] 1.5 Spanning sets & matrix...
 - 6 [Intuitive Deep Learning] 1.6 Generalized matrix...
 - 7 [Deep Learning Matrix Reloaded] Supervised learnin...
 - 8 [Deep Learning Matrix Reloaded] From spanning set...

DGL₁

CLASSIC FEATURE REPRESENTATION
METHODS



AIM

Transform complex features into a space where linear models can work well in the final representation space.

Preliminaries: machine & deep learning, feature extraction and graphs

Personal reflections and notes

Reminder: Supervised regression (linear)

Single regression model

$$l(w) = \frac{1}{n} \sum_{i=1}^n (\tilde{x}_i^T w - y_i)^2$$

(d+1) features

$$\tilde{X} \in \mathbb{R}^{n \times (d+1)} \times w \in \mathbb{R}^{d+1} = \hat{y} \in \mathbb{R}^n \approx y \in \mathbb{R}^n$$

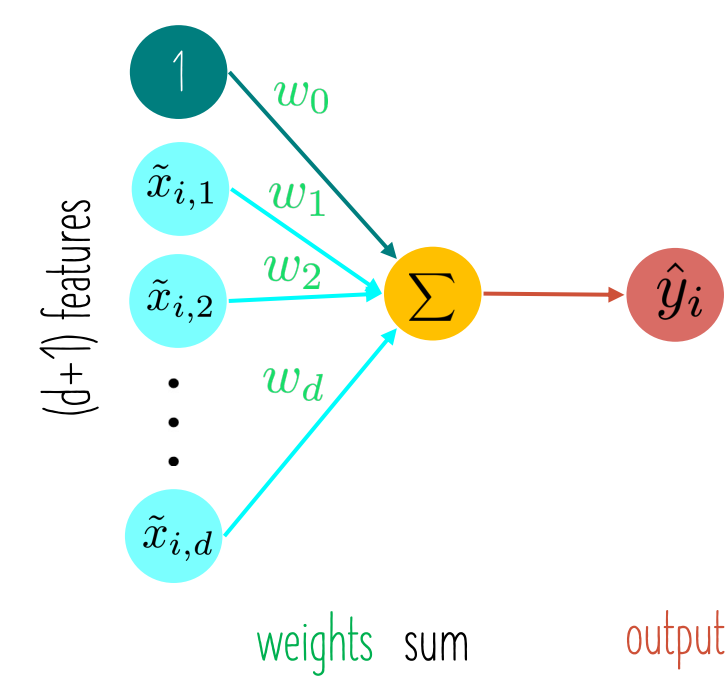
Multi-regression models

$$l(W) = ||\tilde{X}W - Y||_2^2 = \frac{1}{c} \sum_{c=1}^C (\tilde{X}w_c - y_c)^2$$

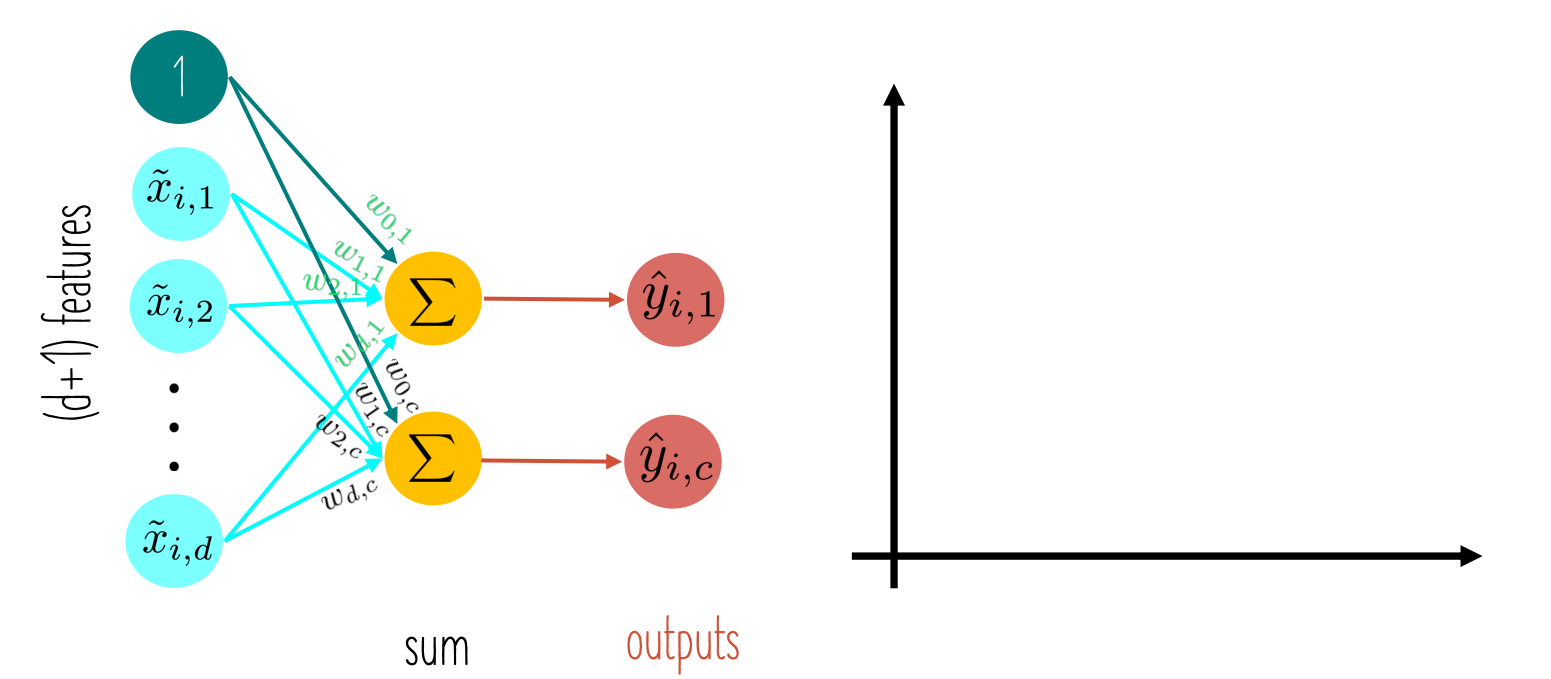
(d+1) features

$$\tilde{X} \in \mathbb{R}^{n \times (d+1)} \times W \in \mathbb{R}^{(d+1) \times C} = \hat{Y} \in \mathbb{R}^{n \times C} \approx Y \in \mathbb{R}^{n \times C}$$

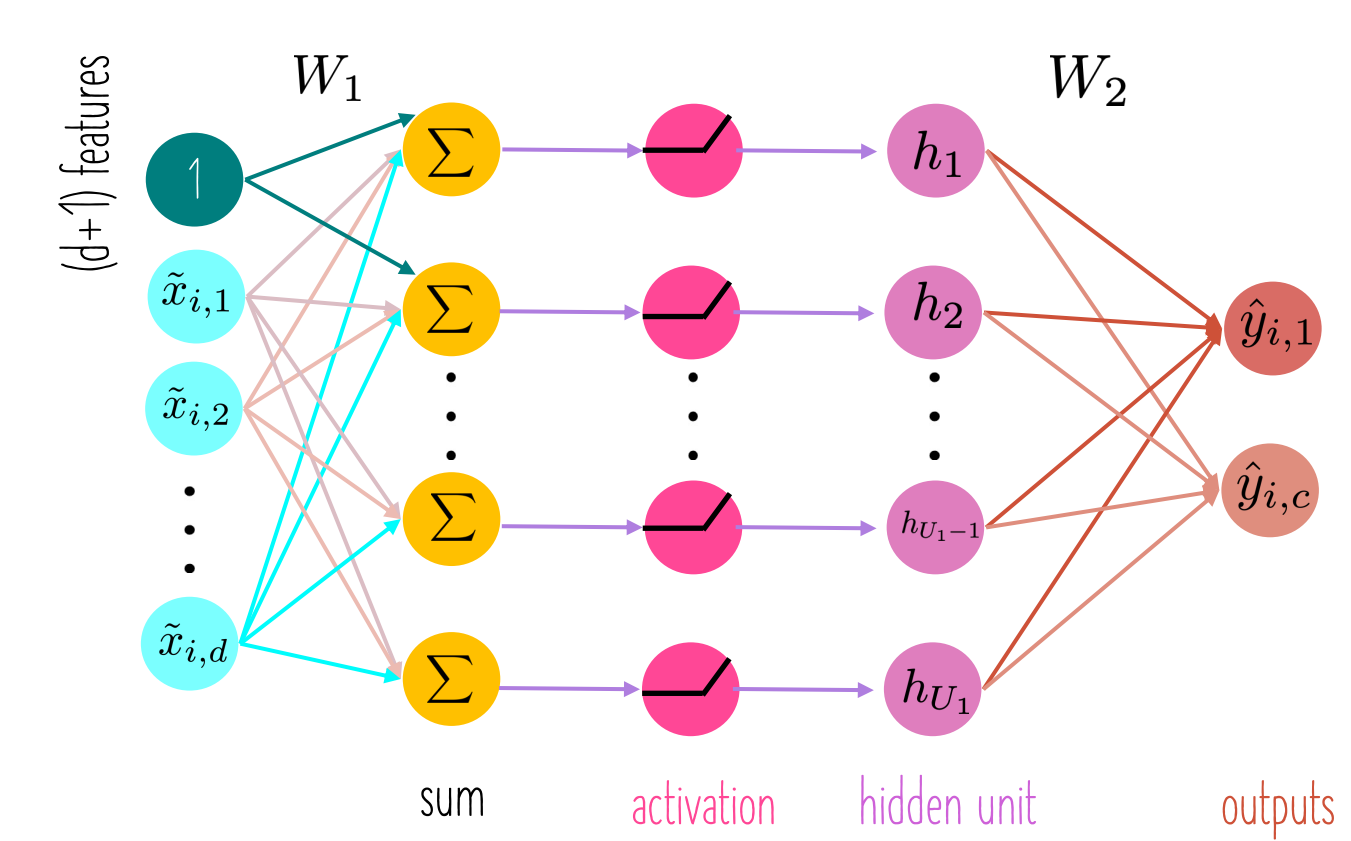
A single layer linear network with a single output



A single layer linear network with a multiple outputs

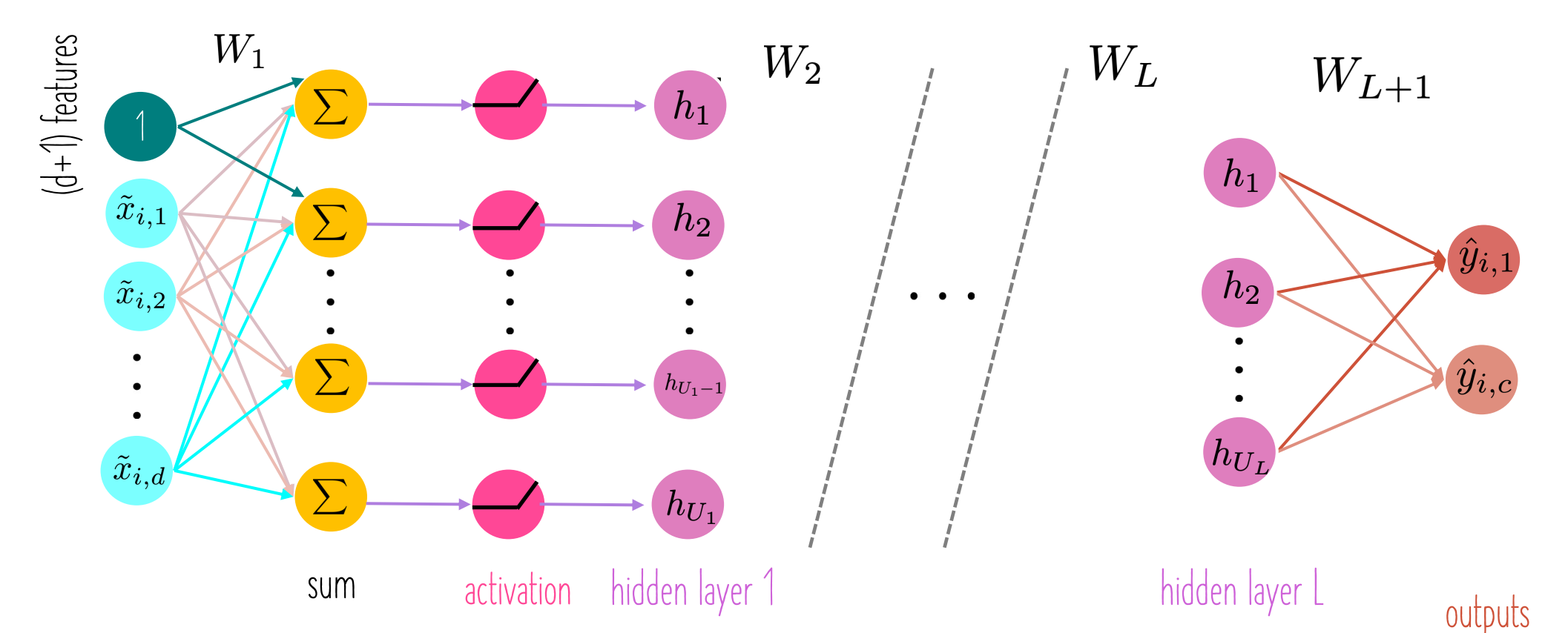


A single layer nonlinear network with a multiple outputs



Let us write down the network mapping function

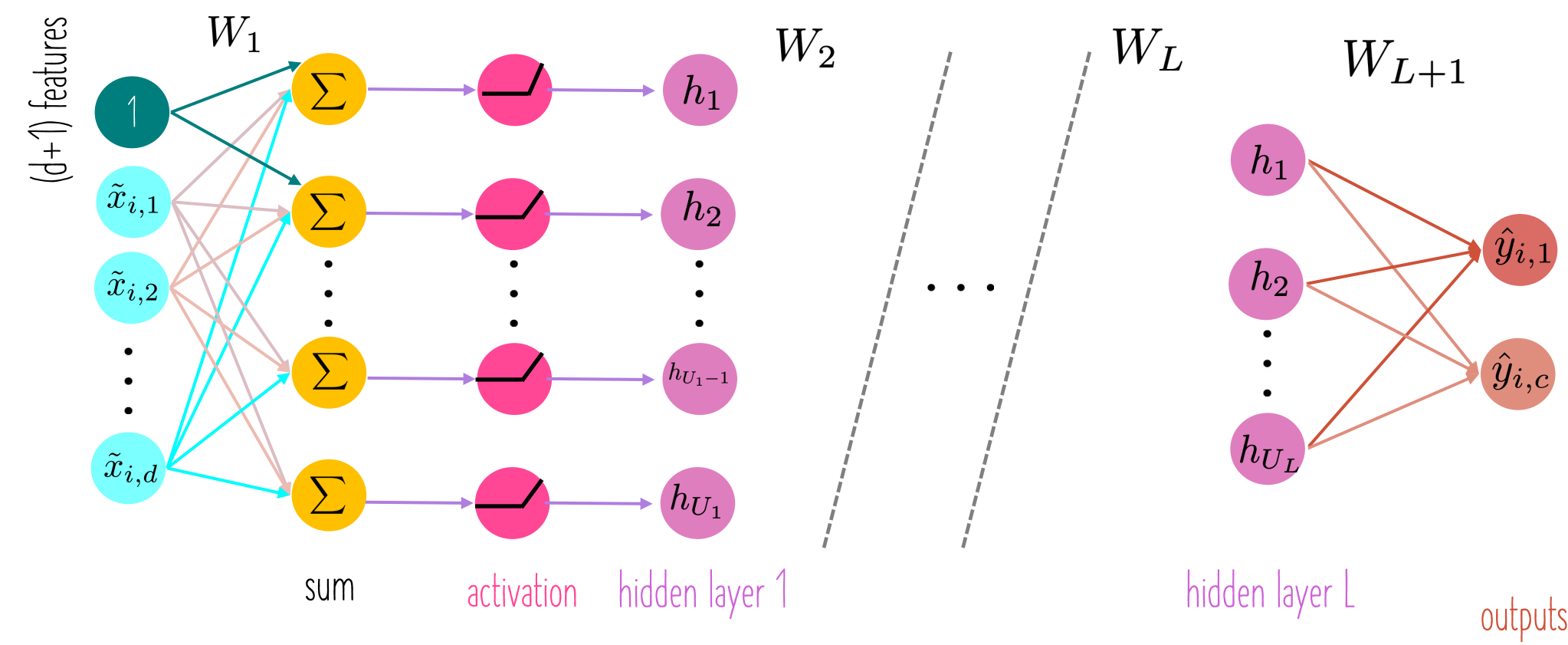
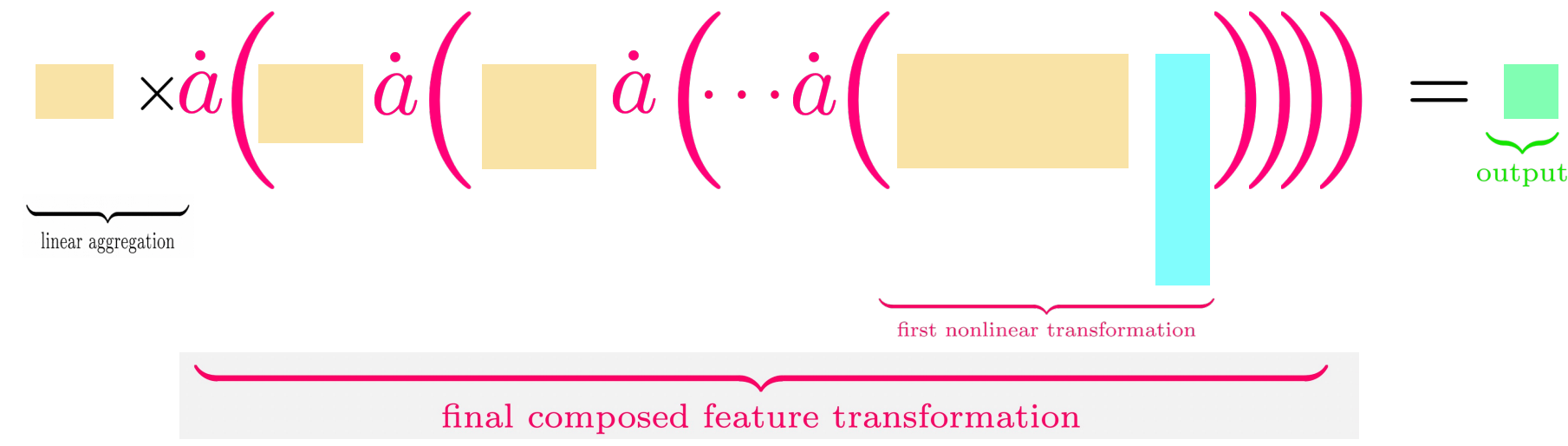
A multi-layer (deep) nonlinear network with a multiple outputs



$$\underbrace{\square}_{\text{linear aggregation}} \times \underbrace{\dot{\square} \left(\underbrace{\square \dot{\left(\square \dot{\left(\dots \dot{\left(\square \right)} \right)} \right)}_{\text{first nonlinear transformation}} \right)}_{\text{final composed feature transformation}} = \underbrace{\square}_{\text{output}}$$

Feature embedding

Feature embedding in deep learning and manifold learning



With higher computational budget we achieve the Da Vinci principle "Simplicity is the Ultimate Sophistication"

😊 The power of DL models lies in learning how to embed complex features sequentially into manifolds (subspaces to learn) that decrease in complexity until 'finding' an ultimate low-dimensional representation space (a much simpler manifold) where linear models can work well.

What is manifold learning?

- ⊗ Underlying reductionist hypothesis: neighborhood preservation across the original and embedding spaces.
- ⊗ The neighborhood in the original domain may be highly noisy (outliers in the local neighborhood of a sample).

Feature embedding

Personal reflections and notes

The evolving landscape of feature embedding

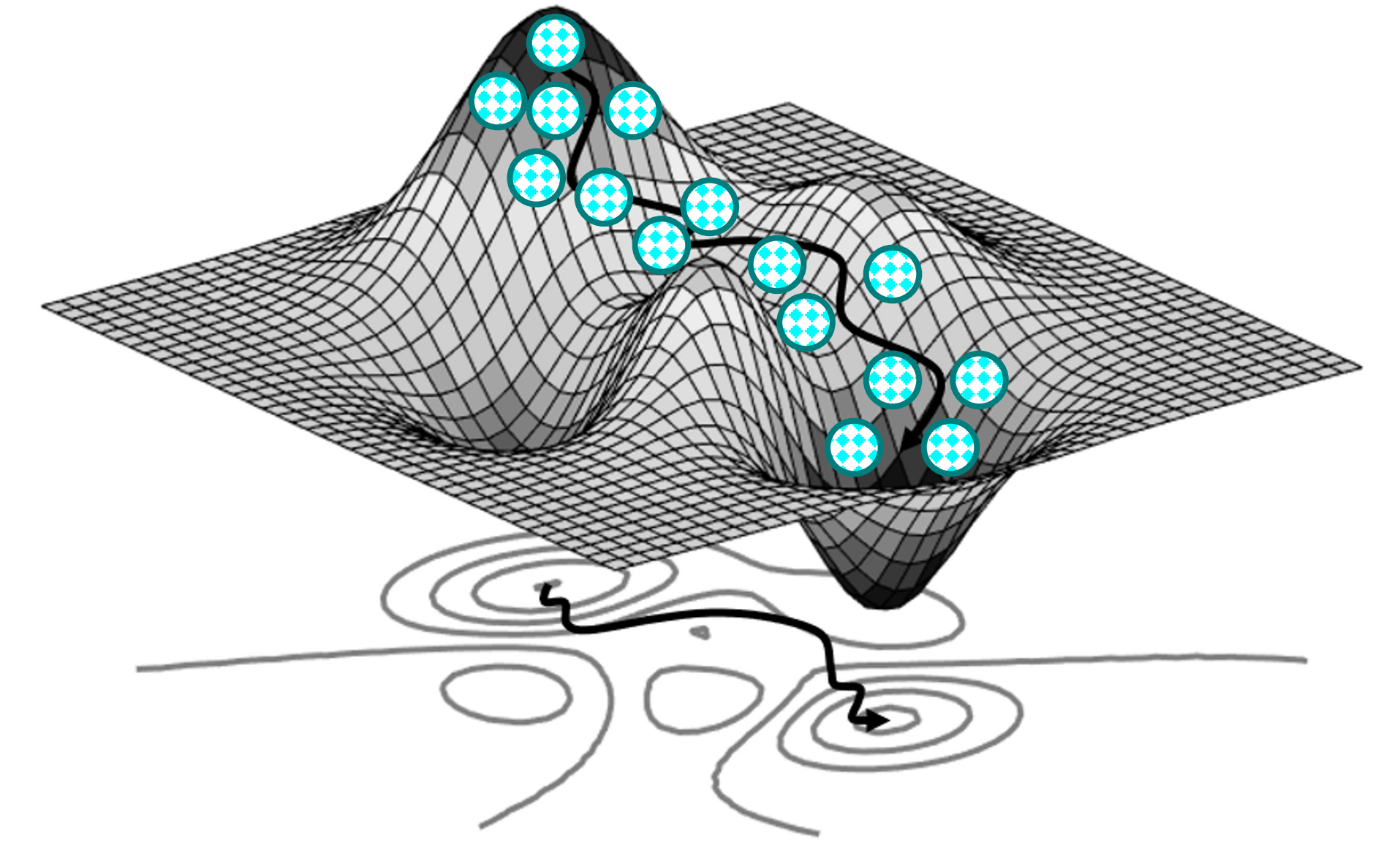
ML problem formalization

$$l(w) = \frac{1}{n} \sum_{i=1}^n (\tilde{x}_i^T w - y_i)^2$$

$\tilde{X} \in \mathbb{R}^{n \times (d+1)}$ $w \in \mathbb{R}^{d+1}$ $\hat{y} \in \mathbb{R}^n$ $y \in \mathbb{R}^n$

We learn a shared mapping w across all training samples.
However, our training samples are not allowed to directly interact.

Individualistic points -- data samples live "independently" on a high-dimensional manifold.



Even in manifold learning, we learn how to calculate the similarity between points but not how to connect them. We are one step away from that!

- ⊗ Cannot utilize node, edge and graph features.
- ⊗ Such approaches cannot obtain embeddings for nodes not in the training set. If we add a node, we need to recomplete all node feature representations (embeddings).

1. Node-based features

2. Edge-based features

3. Graph-based features

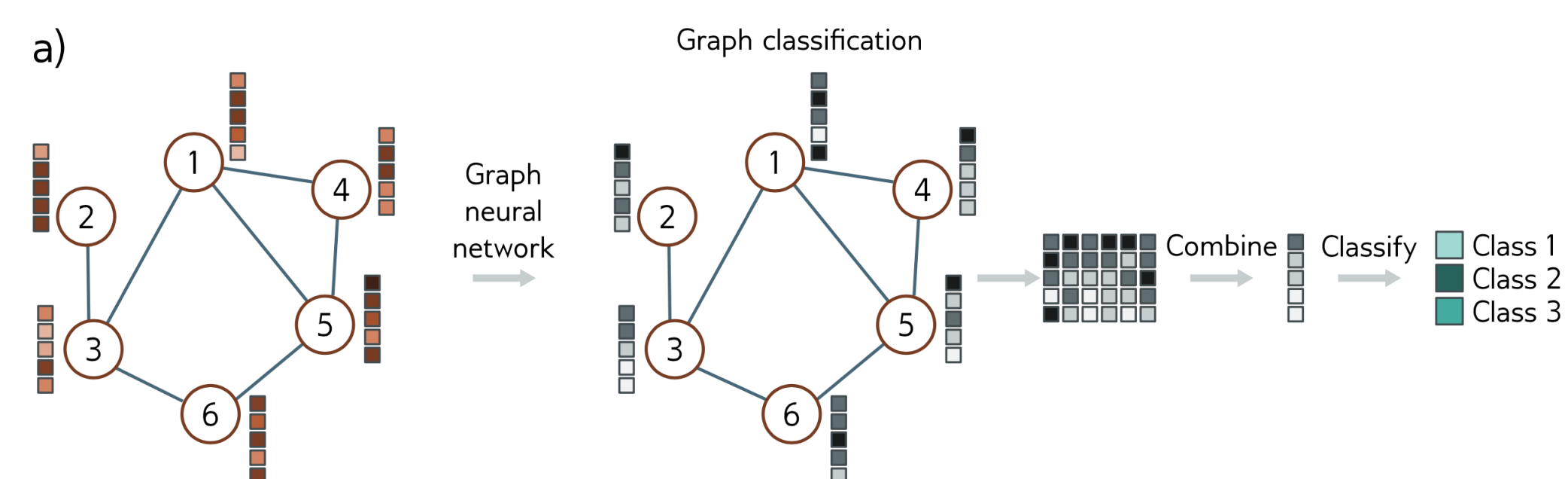
$\tilde{X} \in \mathbb{R}^{n \times (d+1)}$

A

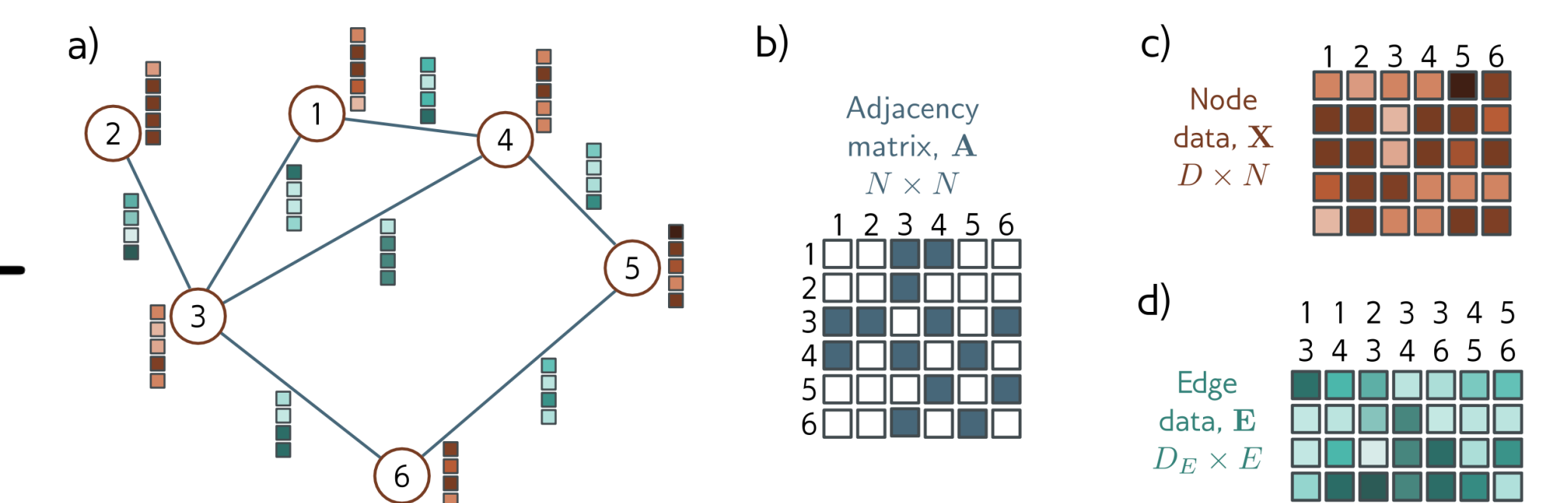
Use a graph to model and capture the relationships between pairs of data points. Each sample interacts with all other samples.

Classical node, edge and graph-based feature extraction methods

Collectivist points -- data samples co-interact and co-learn together via a graph structure (A).



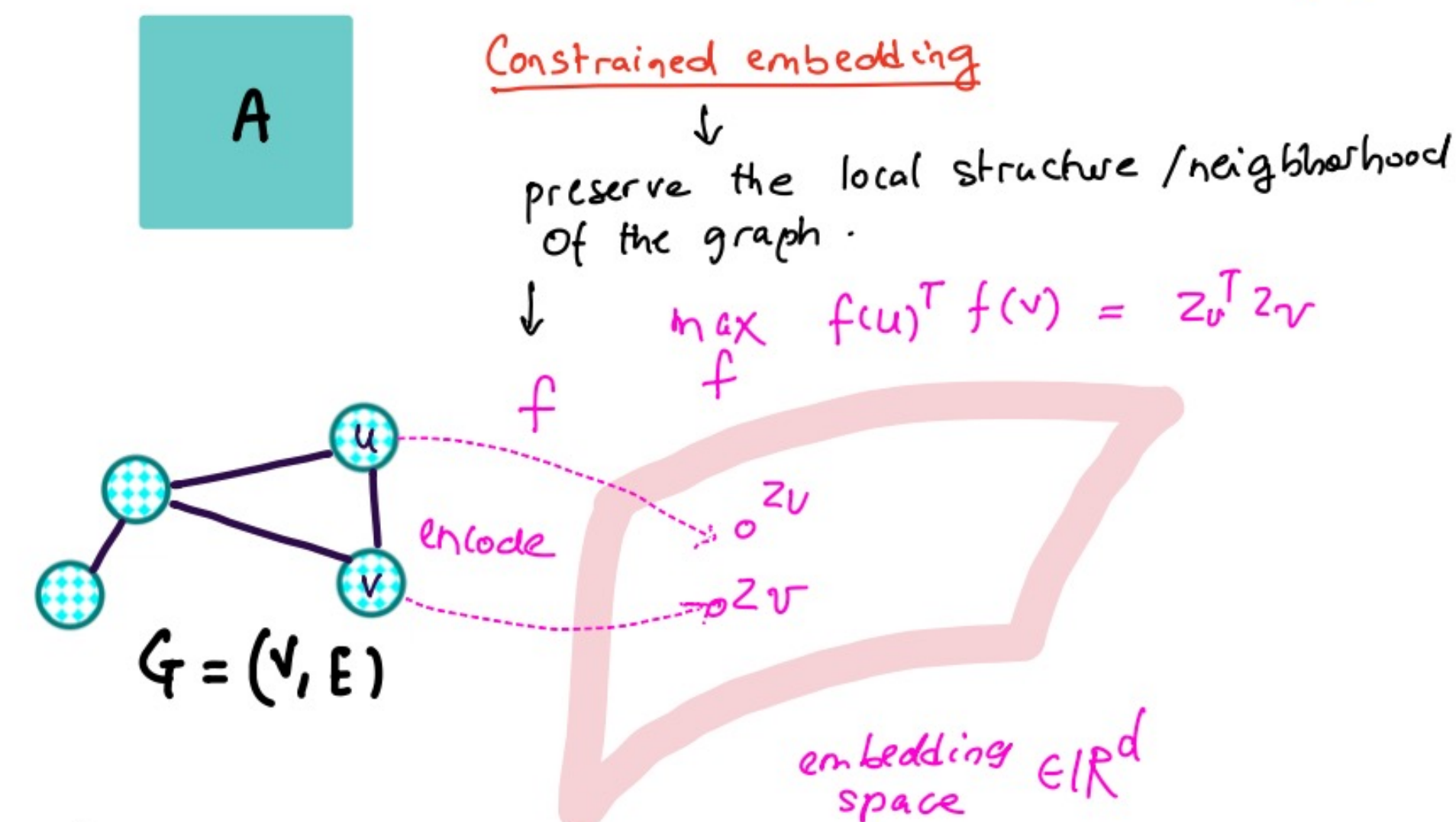
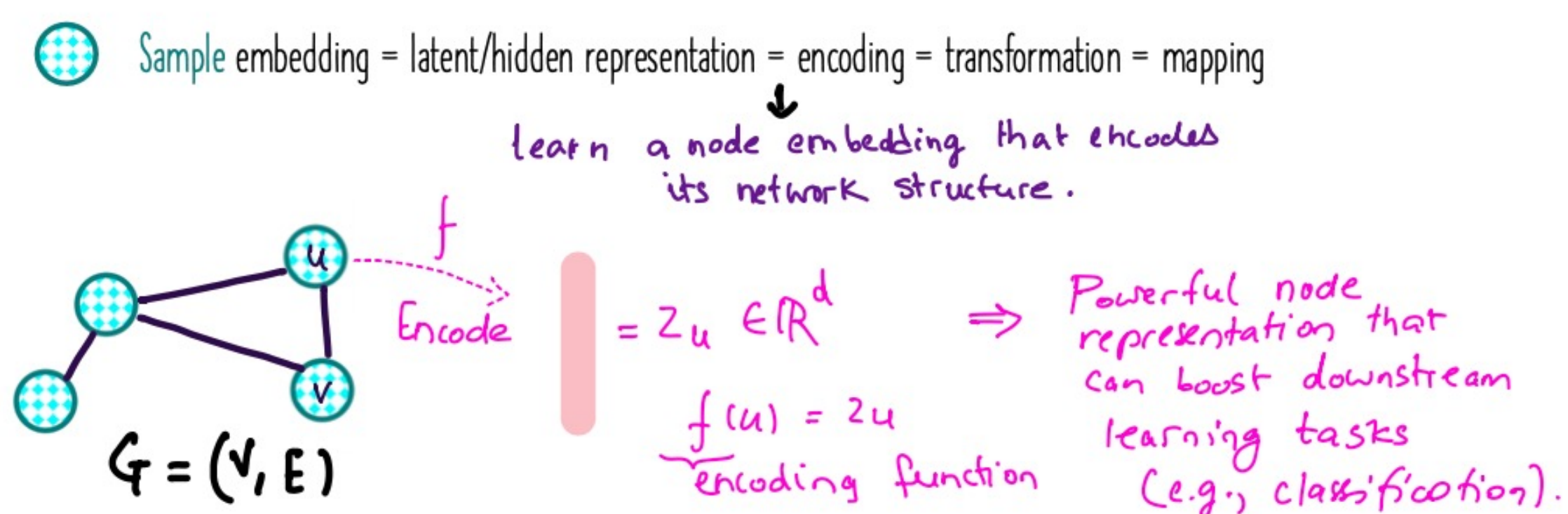
How to get the node features?
How to get the edge or graph features?



We can also add edge features. Remember you've got a graph to learn on!

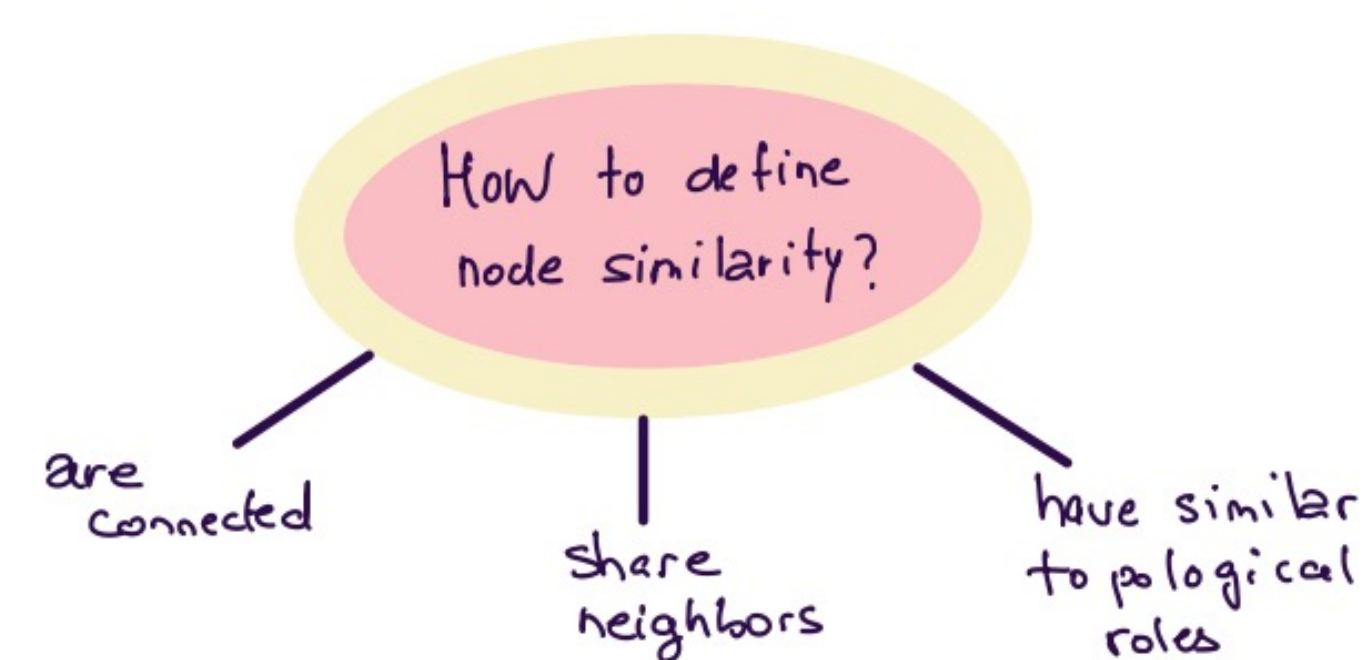
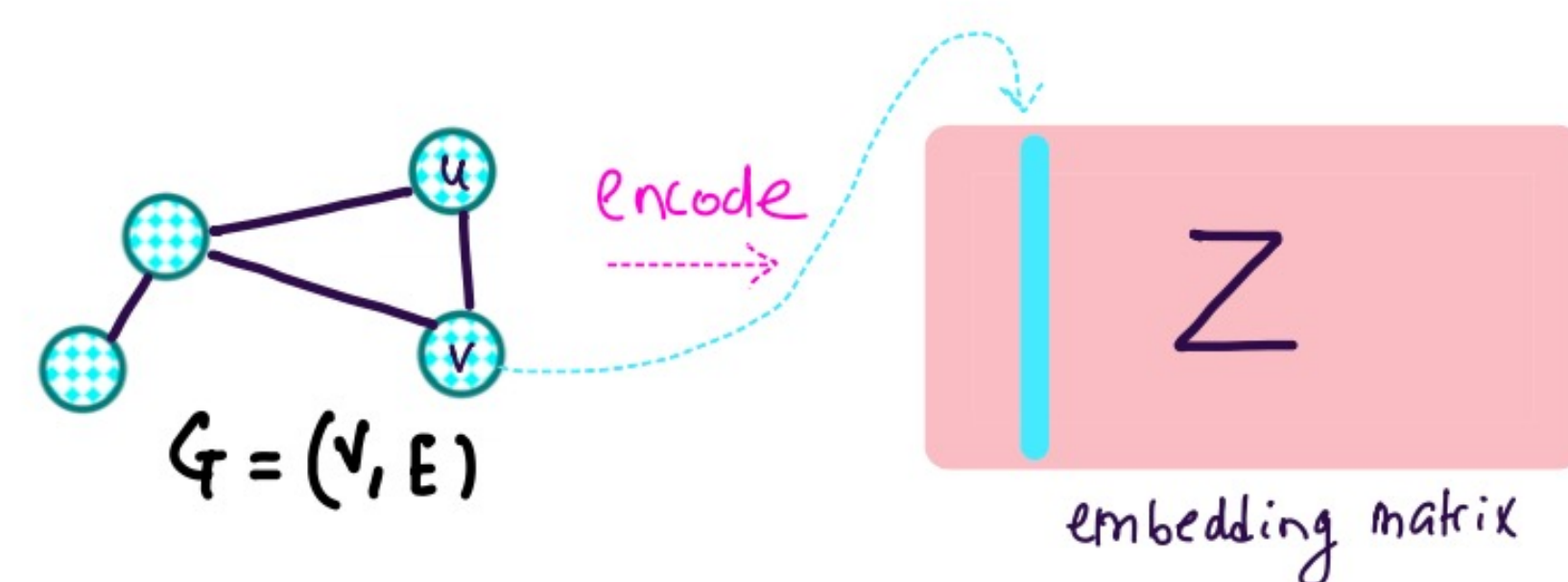
Graph Convolutional Networks (UDL-13)

Classical node encoding/embedding (with no features)



similarity(u, v) $\approx Z_u^T Z_v$

objective: maximize $f(u)^T f(v) = Z_u^T Z_v$
for pairs (u, v) that are close / similar.



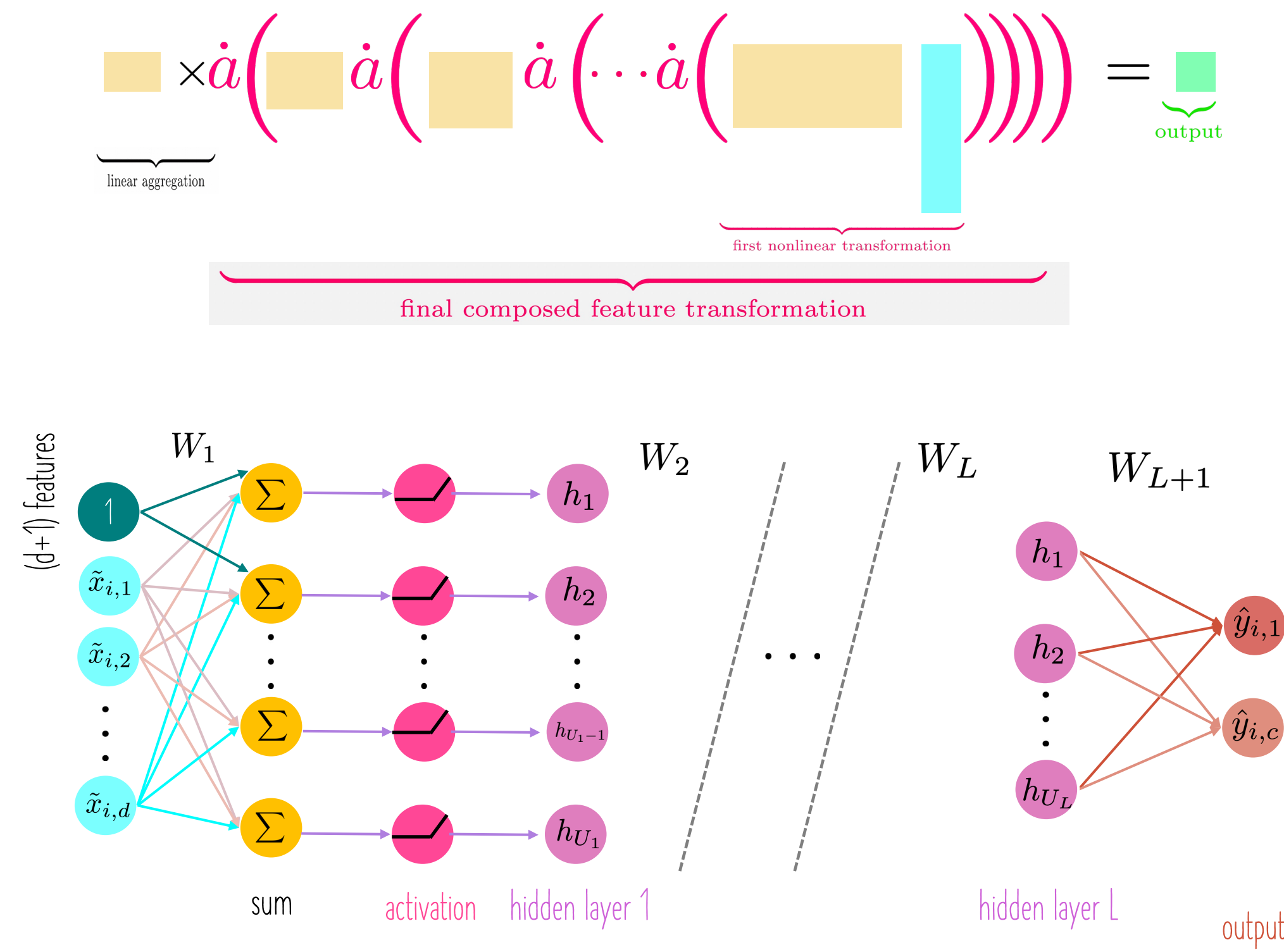
★ TRUE OR FALSE:

- 1) Shallow node embedding is a task-dependent learning problem.
- 2) Node embedding is unsupervised or self-supervised.
- 3) We utilize node labels and features for embedding.

Personal reflections and notes

Graph Convolutional Networks (UDL-13)

A single GCN layer (shallow)

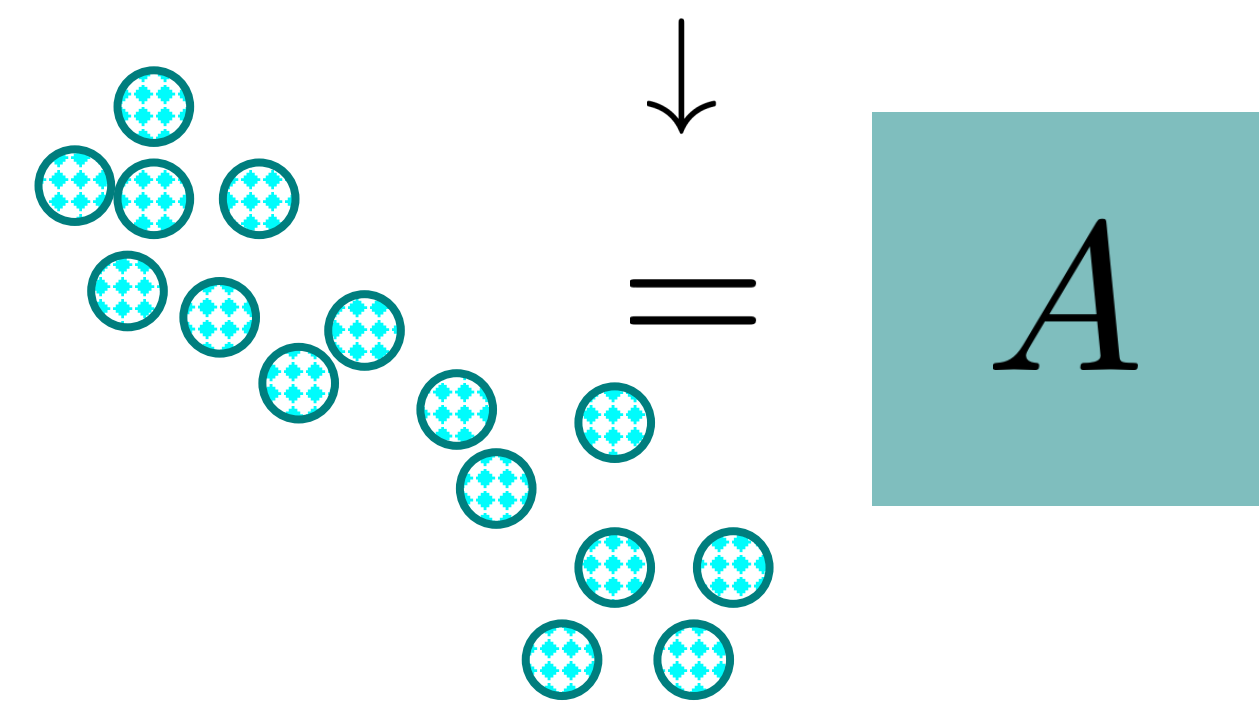


With higher computational budget we achieve the Da Vinci principle "Simplicity is the Ultimate Sophistication"

Our goal: Shifting from an "individualistic" to a "collectivist" learning paradigm through the graph matrix **A**

$$l(w) = \frac{1}{n} \sum_{i=1}^n (\tilde{x}_i^T w - y_i)^2$$

$\tilde{X} \in \mathbb{R}^{n \times (d+1)}$ $w \in \mathbb{R}^{d+1}$ $\hat{y} \in \mathbb{R}^n$ $y \in \mathbb{R}^n$



Deep (sequential) node
representation learning

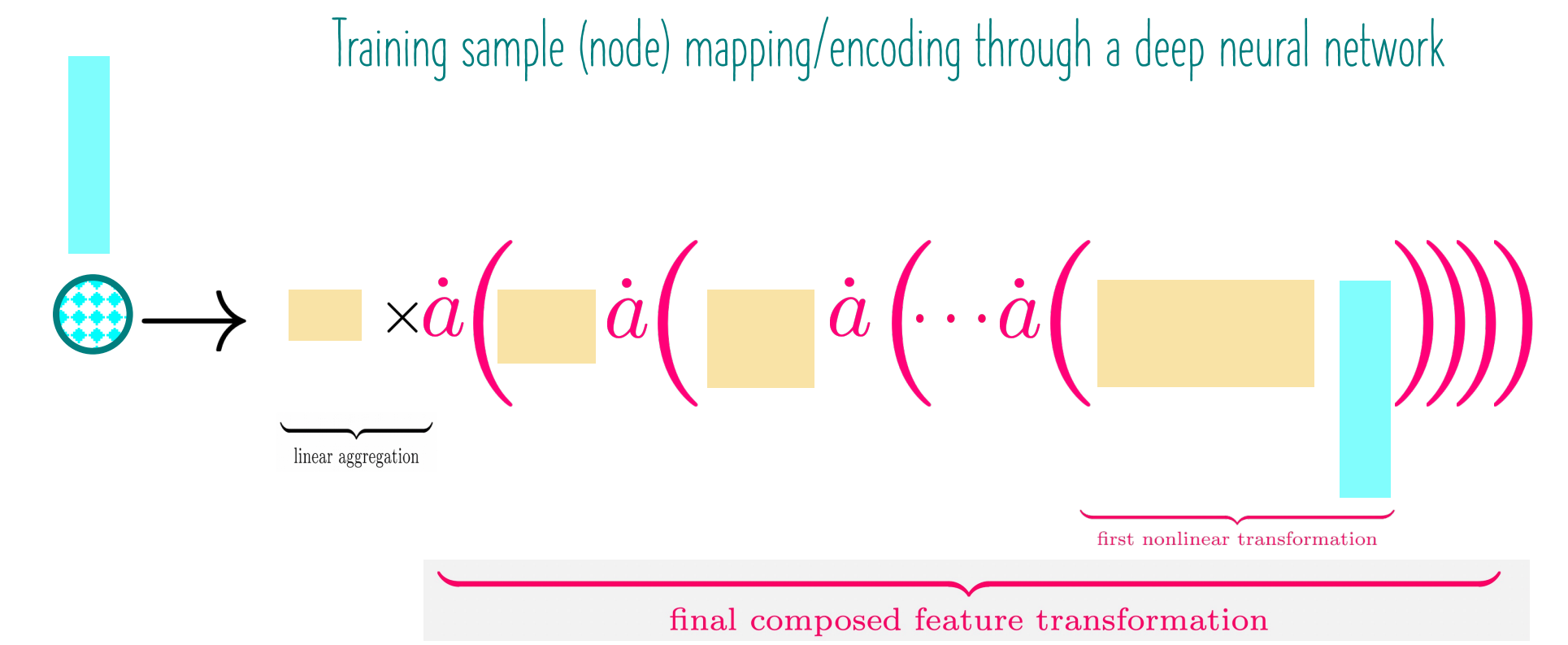
$$\begin{aligned} \mathbf{H}_1 &= \mathbf{F}[\mathbf{X}, \mathbf{A}, \phi_0] \\ \mathbf{H}_2 &= \mathbf{F}[\mathbf{H}_1, \mathbf{A}, \phi_1] \\ \mathbf{H}_3 &= \mathbf{F}[\mathbf{H}_2, \mathbf{A}, \phi_2] \\ &\vdots \\ \mathbf{H}_K &= \mathbf{F}[\mathbf{H}_{K-1}, \mathbf{A}, \phi_{K-1}], \end{aligned}$$

$$l(w) = \frac{1}{n} \sum_{i=1}^n (\tilde{x}_i^T w - y_i)^2$$

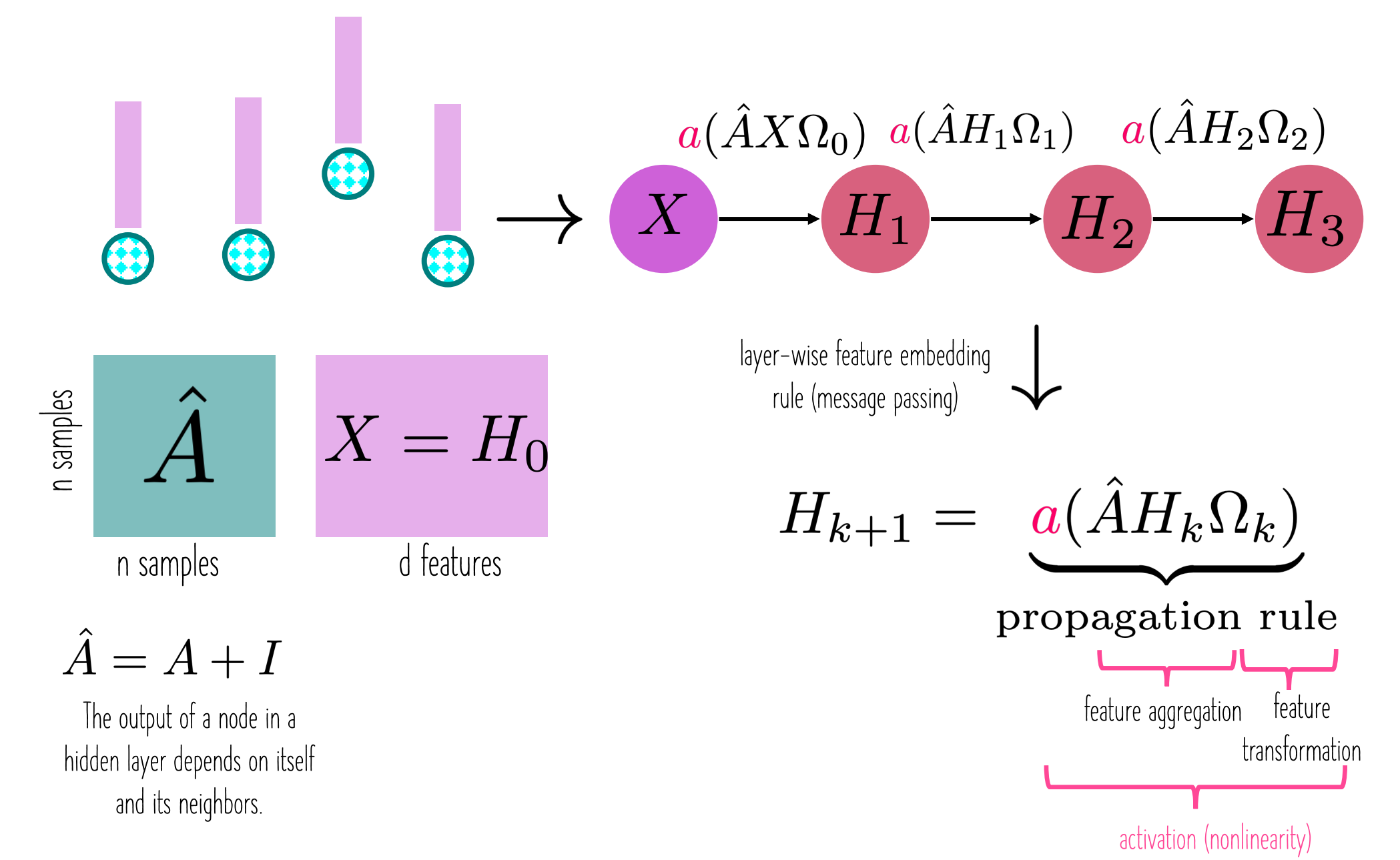
Graph Convolutional Networks (UDL-13)

A single GCN layer (shallow)

A vanilla GCN layer



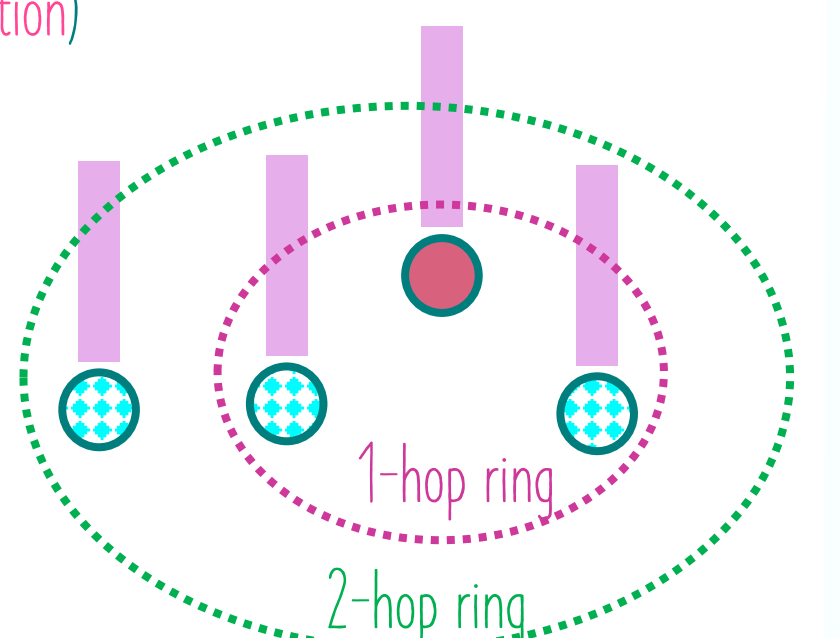
Graph mapping/embedding through a graph neural network



Let us linearize the network by removing the **nonlinearity** (activation)

$$H_1 = \hat{A}H_0\Omega_0$$

$$H_{k+1} =$$

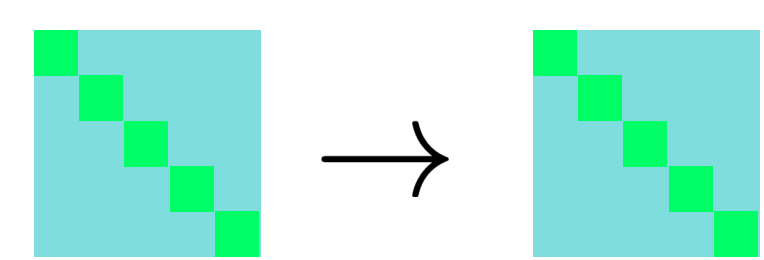


$$\hat{A}^{k+1} =$$


receptive field

Reminder: for each node i , this matrix stores the total number of walks of length $(k+1)$ from node i to a node j in the graph. If you get "0", it means you can't get to j via $(k+1)$ hops or walk.

Layer-wise normalization and propagation rule



$$H_{k+1} = a(\hat{D}^{-1}\hat{A}H_k\Omega_k)$$

normalized propagation rule

$$H_{k+1} = f(H_k, A)$$

generalized propagation rule

- controls the latent embedding variance between layers, which may suppress the gradient explosion or vanishing phenomena.
- such embedding normalization maintains the scale of the output feature vectors.

Graph Convolutional Networks (UDL-13)

Introducing layer-wise non-linearity and biases into GCN

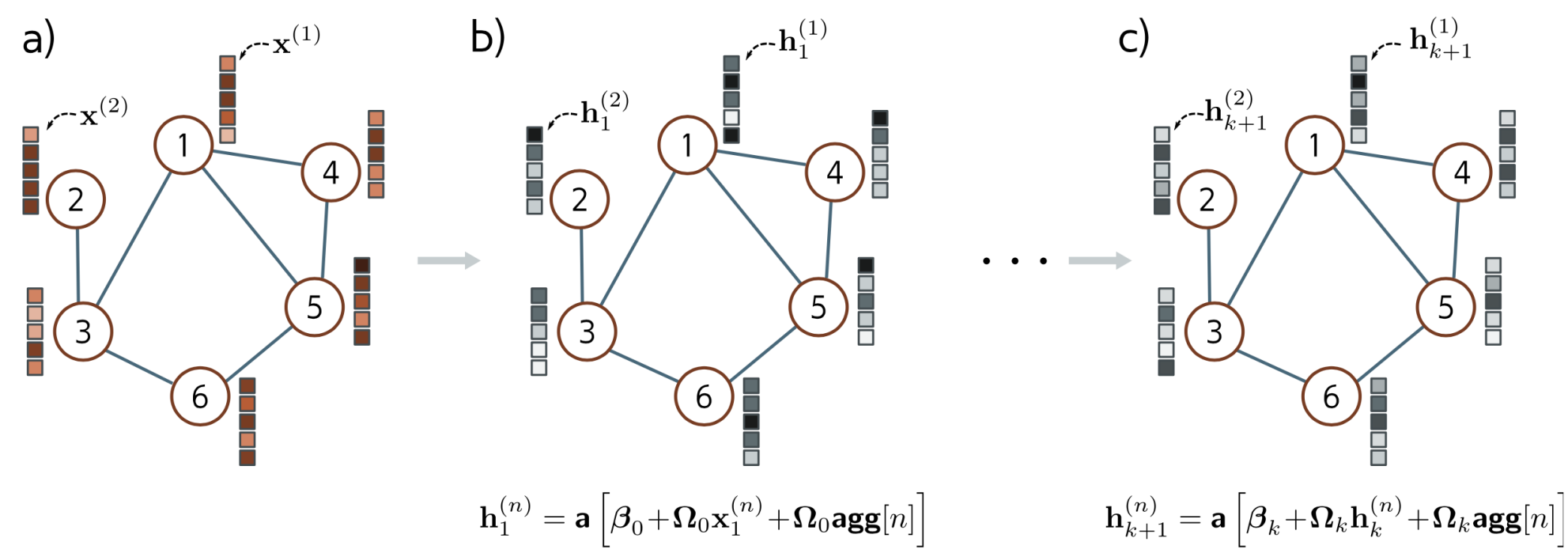


Figure 13.7 Simple Graph CNN layer. a) Input graph consists of structure (embodied in graph adjacency matrix $\mathbf{A}$, not shown) and node embeddings (stored in columns of $\mathbf{X}$). b) Each node in the first hidden layer is updated by (i) aggregating the neighboring nodes to form a single vector, (ii) applying a linear transformation Ω_0 to the aggregated nodes, (iii) applying the same linear transformation Ω_0 to the original node, (iv) adding these together with a bias β_0 , and finally (v) applying a nonlinear activation function $a[\bullet]$ like a ReLU. This process is repeated at subsequent layers (but with different parameters for each layer) until we produce the final embeddings at the end of the network.

Node embedding update rule

message (embedding) aggregation across neighbors

$$\text{agg}[n, k] = \sum_{m \in \text{ne}[n]} h_k^{(m)},$$

$$h_{k+1}^{(n)} = a \left[\beta_k + \Omega_k \cdot h_k^{(n)} + \Omega_k \cdot \text{agg}[n, k] \right]$$

learned embedding of
node n in layer $(k+1)$

$$\begin{aligned} \text{output embedding size} &= a \left[\begin{aligned} &\text{input embedding size} \\ &\text{output embedding size} \end{aligned} \right] \times \text{input embedding size} \\ &+ \begin{aligned} &\text{output embedding size} \\ &\text{input embedding size} \end{aligned} \times \text{input embedding size} \end{aligned}$$

Personal reflections and notes

Graph Convolutional Networks (UDL-13)

Introducing layer-wise non-linearity and biases into GCN

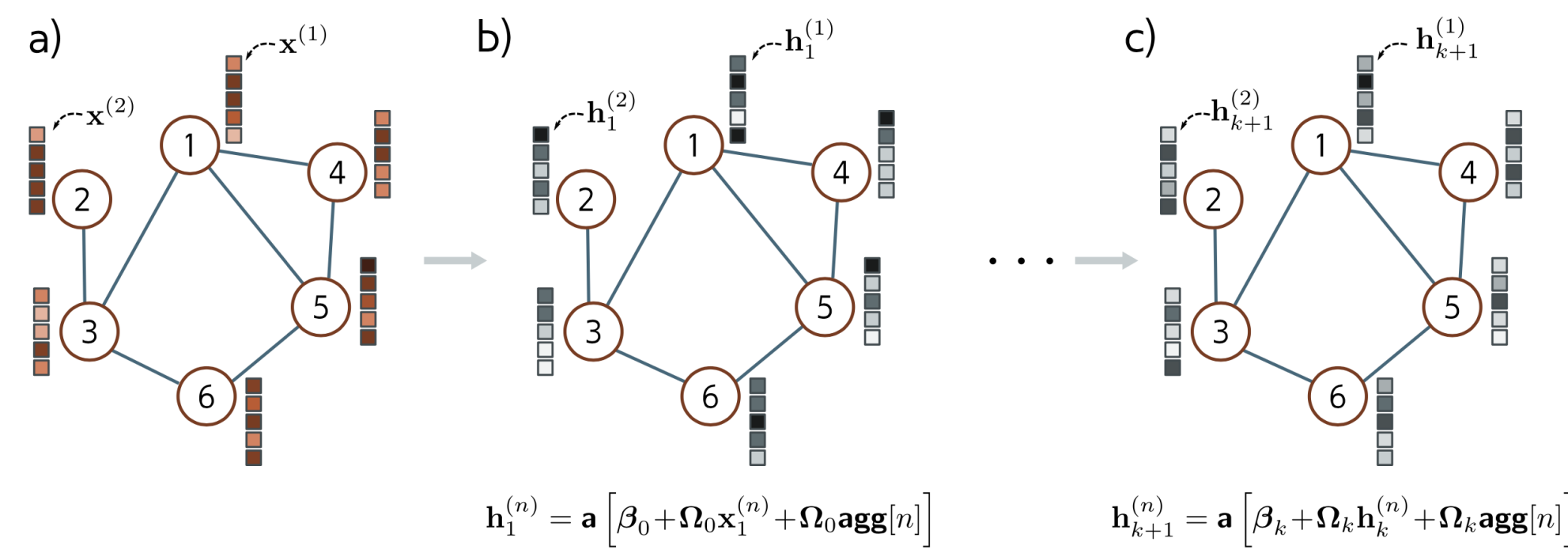


Figure 13.7 Simple Graph CNN layer. a) Input graph consists of structure (embodied in graph adjacency matrix $\mathbf{A}$, not shown) and node embeddings (stored in columns of $\mathbf{X}$). b) Each node in the first hidden layer is updated by (i) aggregating the neighboring nodes to form a single vector, (ii) applying a linear transformation Ω_0 to the aggregated nodes, (iii) applying the same linear transformation Ω_0 to the original node, (iv) adding these together with a bias β_0 , and finally (v) applying a nonlinear activation function $\mathbf{a}[\bullet]$ like a ReLU. This process is repeated at subsequent layers (but with different parameters for each layer) until we produce the final embeddings at the end of the network.

Node embedding update rule

message (embedding) aggregation across neighbors

$$\text{agg}[n, k] = \sum_{m \in \text{ne}[n]} \mathbf{h}_k^{(m)},$$

$$\mathbf{h}_{k+1}^{(n)} = \mathbf{a} \left[\beta_k + \Omega_k \cdot \mathbf{h}_k^{(n)} + \Omega_k \cdot \text{agg}[n, k] \right]$$

learned embedding of
node n in layer $(k+1)$

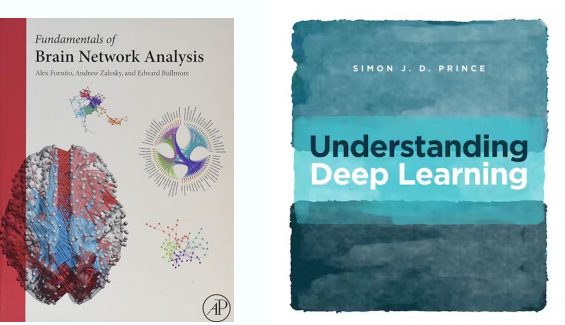
$$\begin{aligned} \text{output embedding size} &= \mathbf{a} \left[\begin{aligned} &\text{output embedding size} \\ &+ \text{output embedding size} \cdot \dots \cdot \text{output embedding size} \times \text{input embedding size} \\ &+ \text{output embedding size} \cdot \dots \cdot \text{output embedding size} \times \text{input embedding size} \end{aligned} \right] \end{aligned}$$

Update rule for all nodes

$$\begin{aligned} \mathbf{H}_{k+1} &= \mathbf{a} \left[\beta_k \mathbf{1}^T + \Omega_k \mathbf{H}_k + \Omega_k \mathbf{H}_k \mathbf{A} \right] \\ &= \mathbf{a} \left[\beta_k \mathbf{1}^T + \Omega_k \mathbf{H}_k (\mathbf{A} + \mathbf{I}) \right], \end{aligned}$$

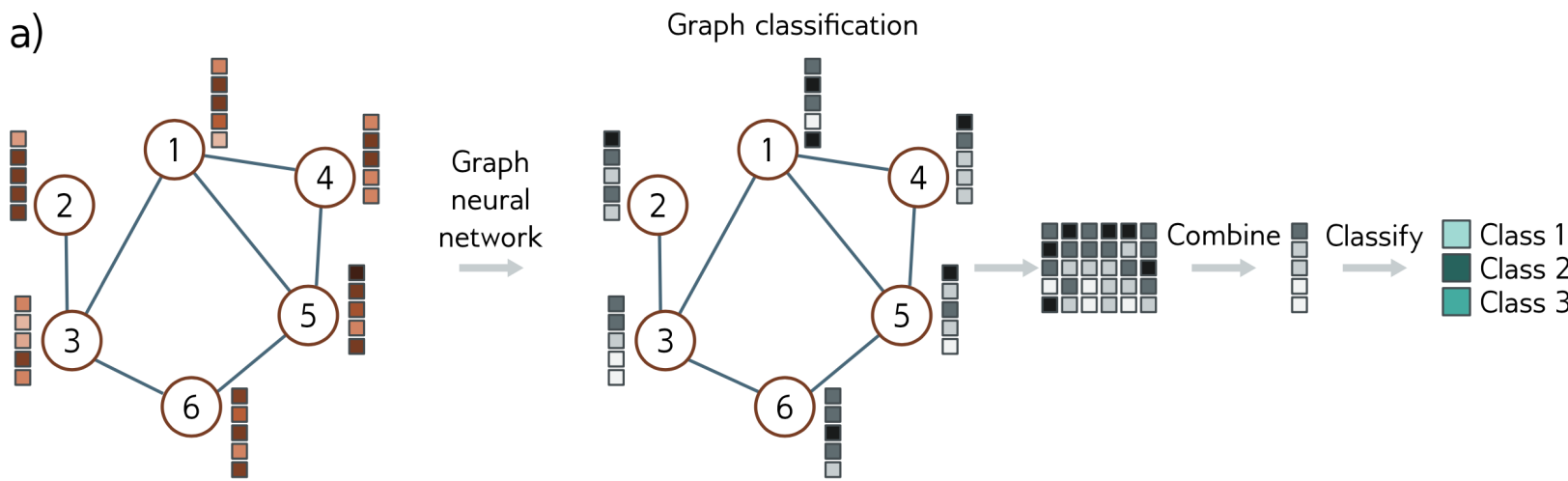
$$\begin{aligned} \text{output embedding size} \times \text{nodes} &= \mathbf{a} \left[\text{output embedding size} \times \text{nodes} \right] \\ &+ \left[\begin{aligned} &\text{self-encoding} \\ &\text{neighborhood encoding} \end{aligned} \right] \end{aligned}$$

Personal reflections and notes



Graph Convolutional Networks (UDL-13)

Example: graph classification



Deep (sequential) node
representation learning

$$\begin{aligned} \mathbf{H}_1 &= \mathbf{a} [\beta_0 \mathbf{1}^T + \Omega_0 \mathbf{X}(\mathbf{A} + \mathbf{I})] \\ \mathbf{H}_2 &= \mathbf{a} [\beta_1 \mathbf{1}^T + \Omega_1 \mathbf{H}_1(\mathbf{A} + \mathbf{I})] \\ &\vdots \\ \mathbf{H}_K &= \mathbf{a} [\beta_{K-1} \mathbf{1}^T + \Omega_{K-1} \mathbf{H}_{K-1}(\mathbf{A} + \mathbf{I})] \end{aligned}$$

Network output function (output prediction)

$$\begin{aligned} \mathbf{f}[\mathbf{X}, \mathbf{A}, \Phi] &= \text{sig} [\beta_K + \omega_K \mathbf{H}_K \mathbf{1} / N] \\ y &= \text{sig} [\beta_K + \omega_K \mathbf{H}_K \mathbf{1} / N] \end{aligned}$$

$$\begin{array}{c} \text{learned weights} \\ \text{embedding size} \end{array} \times \begin{array}{c} \text{nodes} \\ \vdots \end{array} \times \begin{array}{c} \text{"1" vector} \\ \frac{1}{N} \end{array} = \text{output}$$

Going deeper by adding more layers

How to combine the current node and aggregated neighbors?

Simply sum them up

$$\mathbf{H}_{k+1} = \mathbf{a} [\beta_k \mathbf{1}^T + \Omega_k \mathbf{H}_k(\mathbf{A} + \mathbf{I})]$$

Add a diagonal enhancement

$$\mathbf{H}_{k+1} = \mathbf{a} [\beta_k \mathbf{1}^T + \Omega_k \mathbf{H}_k(\mathbf{A} + (1 + \epsilon_k) \mathbf{I})]$$

Going deeper with more layers

Treat the current node differently from the aggregated neighbors by applying a different linear transform Ψ_k to the current node:

$$\begin{aligned} \mathbf{H}_{k+1} &= \mathbf{a} [\beta_k \mathbf{1}^T + \overbrace{\Psi_k \mathbf{H}_k} + \overbrace{\Omega_k \mathbf{H}_k \mathbf{A}}] \\ &= \mathbf{a} \left[\beta_k \mathbf{1}^T + [\Omega_k \quad \Psi_k] \begin{bmatrix} \mathbf{H}_k \mathbf{A} \\ \mathbf{H}_k \end{bmatrix} \right] \\ &= \mathbf{a} \left[\beta_k \mathbf{1}^T + \Omega'_k \begin{bmatrix} \mathbf{H}_k \mathbf{A} \\ \mathbf{H}_k \end{bmatrix} \right], \end{aligned}$$

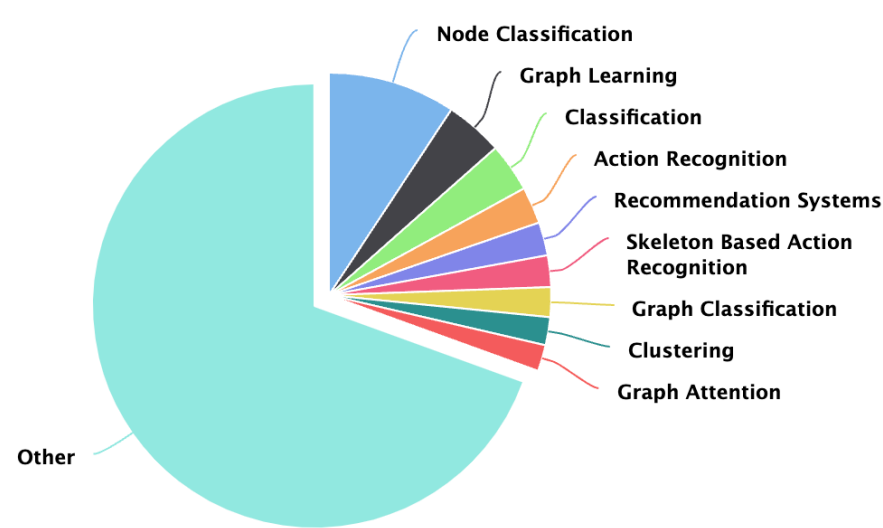


Graph Convolutional Networks (UDL-13)

GCNs are hot topics!

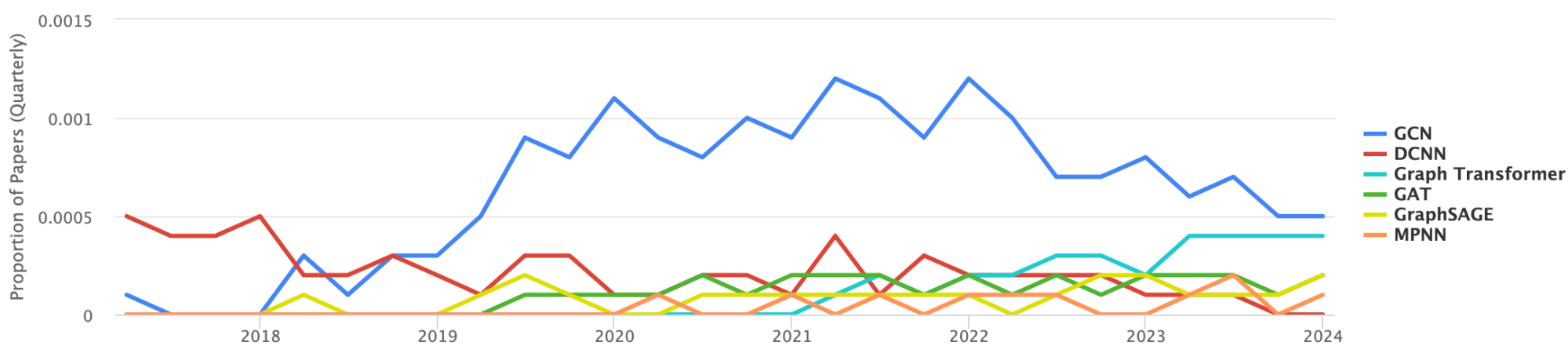
Papers with code

Tasks



Task	Papers	Share
Node Classification	73	9.29%
Graph Learning	33	4.20%
Classification	28	3.56%
Action Recognition	21	2.67%
Recommendation Systems	19	2.42%
Skeleton Based Action Recognition	18	2.29%
Graph Classification	17	2.16%
Clustering	16	2.04%
Graph Attention	15	1.91%

Usage Over Time



⚠ This feature is experimental; we are continuously improving our matching algorithm.

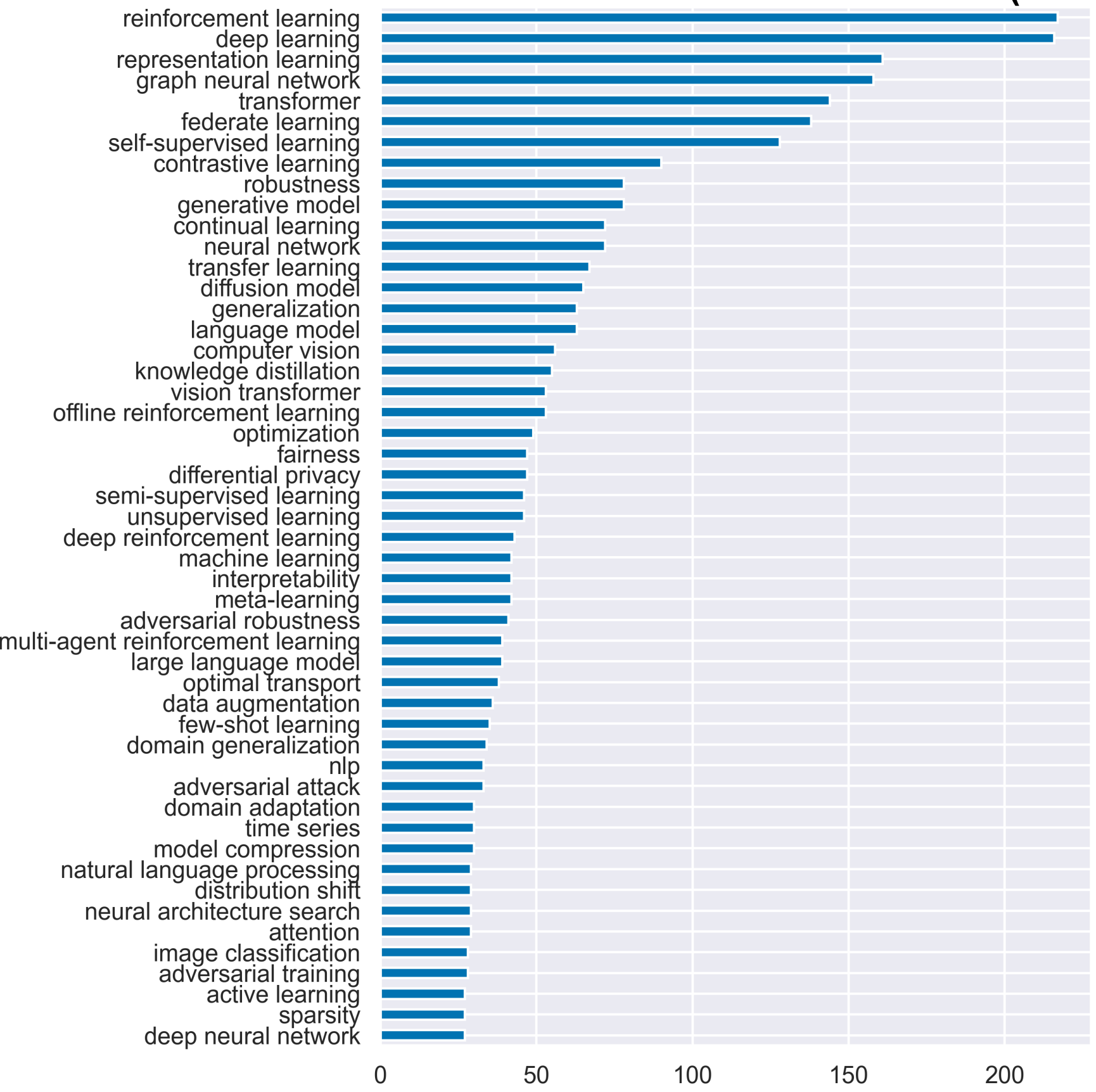
<https://paperswithcode.com/method/gcn>

ICLR 2023 open review data

- Top-10 Ranking between 2022 and 2023 (full list please refer to [keywords.md](#))

Keyword	2022	2023
reinforcement learning	1	1
deep learning	2	2
representation learning	4	3
graph neural network	3	4
transformer	5	5
federate learning	7	6
self-supervised learning	6	7
contrastive learning	10	8
robustness	9	9
generative model	8	10

50 MOST APPEARED KEYWORDS (2023)



<https://github.com/EdisonLeeeee/ICLR2023-OpenReviewData/blob/main/ov-file>

Recall box: summarize what you remember from this lecture below